2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) | 979-8-3503-3066-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/SANER60148.2024.00041

Navigating Confidentiality in Test Automation: A Case Study in LLM Driven Test Data Generation

1st Hrishikesh Karmarkar
TCS Research
Mumbai, India
hrishi.karmarkar@tcs.com

2nd Supriya Agrawal
TCS Research
Pune, India
supriya.agrawal1@tcs.com

3rd Avriti Chauhan*
TCS Research
Pune, India
avriti.p.chauhan@gmail.com

4th Pranav Shete
TCS Research
Pune, India
pranav.shete@tcs.com

***Abstract*—In outsourced industrial projects for testing of web applications, often neither the application to be tested, nor its source code are provided to the testing team, due to confidentiality reasons, making systematic testing of these applications very challenging. However, textual descriptions of such systems are often available. So, one can consider leveraging a Large Language Model (LLM) to parse these descriptions and synthesize test generators (programs that produce test data). In our experience, LLM synthesized test generators suffer from two problems:- (1) unsound: the generators might produce invalid data and (2) incomplete: the generators typically fail to generate all expected valid inputs. To mitigate these problems, we introduce *TestRefineGen* a method for autonomously generating test data from textual descriptions. *TestRefineGen* begins by invoking an LLM to parse a given corpus of documents and produce multiple test generators. It then uses a novel ranking approach to identify generators that can produce invalid test data, and then automatically repairs them using a counterexample-guided refinement process. Lastly, *TestRefineGen* performs a generalization procedure that offsets synthesis or refinements that leads to incompleteness, to obtain generators that produce more comprehensive valid inputs.**

We evaluated the effectiveness of *TestRefineGen* on a manually curated set of 256 textual descriptions of test data. *TestRefineGen* synthesized generators that produce valid test data for 66.01 % of the descriptions. Using a combination of post-processing sanitisation and refinement it was able to successfully repair synthesized generators, which improved the success rate to 76.95 %. Further, our statistical analysis on a small subset of synthesized generators shows that *TestRefineGen* is able to generate test data that is well distributed across the input space. Thus, *TestRefineGen* can be an effective technique for autonomous test data generation for web testing in projects with confidentiality concerns.

***Index Terms*—automated test input generation, testing, refinement, Large Language Models**

I. Introduction

Web applications are one of the key drivers of critical activities in business domains like banking, finance and insurance. This reliance on applications running in browsers across various devices, necessitates their reliable and correct functioning, which is a complicated task due to the continuously evolving landscape of web technologies [1].

Typical automated testing tools for web applications allow developers to write test code that generates data and interacts with GUI elements, thereby streamlining the testing process. This approach is particularly advantageous when frequent changes to applications necessitate updates to test suites and mitigates many complications associated with manual testing that often lead developers to forego the creation of test suites [2]. Yet generating tests that are accurate, meaningful and exercise real world combinations remains a challenge [3].

Insights about best practices for web test automation from the grey literature confirm that developers often adopt data-driven testing techniques by parameterizing test cases with accurate and meaningful real-world combinations of test input data [4]. Also, it is well known that the economic viability of test automation is related to the cost of maintaining test suites [5], which in turn depends critically on sound management of test data. Thus, test data crucially affects both the quality and economic viability of an automatically generated test-suite for web application testing.

In our experience of consulting with customers of TCS's web application testing tool [6], a recurring requirement that comes forth is: many customers, who invest in test automation tools for web applications, seek ways to enhance these tools' effectiveness through our research and innovation, *without sharing their web applications or source code for testing, as these contain confidential information.* In the absence of a *box* to test, neither black-box testing techniques (which require the application executable), nor white box testing techniques (which require source code), nor grey box testing (combination of black box and white box testing) techniques are suitable. We call this as *no-box testing*, as no box (white, black, or grey) is available during test generation, due to the *confidentiality of access* problem.

There exist test generation techniques for no-box testing. These typically require either a formal specification of the system [7] or an initial partial test suite with each test marked as positive or negative. In an industrial setting, where products such as the TCS's web application

*This work was done when Avriti was with TCS Research.

2640-7574/24/$31.00 ©2024 IEEE
DOI 10.1109/SANER60148.2024.00041

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

testing tool need to compete against similar offerings from other vendors to retain customers, the confidential access problem profoundly shapes the approach to software testing and automated test data generation. Thus, any no-box testing innovation in this space that is cognizant of the confidential access requirement needs to exhibit the following key characteristics:

- Text driven Test Generation: The technique should be capable of generating test data based solely on textual descriptions like user stories, user manuals, high-level requirement documents, FAQs, and client agreements.
- Robust Test Data Validation and Repair Mechanisms: as test data is generated with no box and no formal requirements, it can often be ill formed. Therefore, there should be a robust validation mechanism to ensure that the test data is well formed. In case some generated test data is found to be invalid, the technique should be able to repair or refine the generated test data.
- Scalability and efficiency: The approach should be scalable to handle large volumes of test data generation without significant degradation in performance or accuracy, while keeping the test generation costs under control.

There has been a growing belief that machine learning techniques, especially those in the realm of natural language processing (NLP), can be used to address these requirements and the involved challenges. This has led to a surge of interest in applying techniques that use pretrained transformer models like BART [8] to the problem of automated test generation [9]. This has been further fuelled by the development of large language models (LLM) like GPT [10] leading to the development of even more sophisticated techniques [9], [11], [12]. In large part, this belief is not without foundation as LLMs have been shown to perform well in code generation [13] and hence can be applied directly to the generation of test automation scripts and libraries. Also, LLMs are known to be strong at commonsense text inference [14] and can be deployed to mitigate some problems associated with specifications that are incomplete or described in natural language when generating test automation scripts and libraries.

However, current state-of-the-art techniques that use LLMs either address the problem of automated unit test generation without emphasizing the test data quality problem or emphasize it in restricted settings like smartphone mobile applications. More importantly, none of these techniques adequately deal with the confidential access problem in a no-box settting. This leaves a significant gap in the state-of-the-art.

In this paper, we address the above gap with *TestRefineGen*, our novel approach to automated LLM based test data generation for web applications that addresses the challenges posed by confidentiality constraints in the no-box setting. *TestRefineGen* uses Large Language Models (LLM) to first produce *generator* Python programs, which when executed generate test data. These programs are generated by prompting a LLM with textual descriptions (including possibly ambiguous natural language descriptions) of the test data, derived from various sources like user stories, FAQs, and client agreements. By choosing to use a LLM to synthesize Python generator programs rather than repeatedly query the LLM to generate tests [11], [12] *TestRefineGen* can generate test data automatically at scale and limit the costs involved in accessing the LLM to the initial phase of synthesizing the generator programs. Also, clients are readily willing to deploy packaged generator programs as a plugin module for an existing test automation tool, which makes it a very important consideration for us.

Another aspect of automatically generated test data is that it can be either valid i.e. it satisfies structural constraints given by a specification or invalid. It is important to know if test data is valid or invalid *a priori* to its use in testing. Because, test data that is known to be valid is useful in testing as high quality seed inputs in fuzzing based techniques or as data for crafting high quality test-cases in functional testing. However, invalid inputs are equally useful since error-handling code for invalid inputs can often contain more bugs than regular code [15]. Also, a priori knowledge of invalid inputs can help in mitigating masking of other inputs during combinatorial testing [16]. To address the issue of test data validity, *TestRefineGen* also generates a set of *validator* Python programs which validate the generated test data, using the same textual descriptions used to synthesize the generators. Each generator is subsequently *ranked* for further post-processing using a scheme, based on the Pass@k metric [13]. If a generator is expected to generate valid data, but the ranking classifies it as one that generates invalid data, *TestRefineGen* attempts to repair or refine it using a counter-example guided strategy inspired by the CEGAR [17] technique, which is used extensively in formal program verification. The refinement and repair also partially address the issue of ambiguous or incomplete specifications by post-processing of the LLM synthesized generators and test data.

A potential problem that can arise when encoding constraints extracted from textual test data descriptions as a generator program is that the set of constraints can be too strong [18]. In this case, the generator can output only a subset of all expected valid test data. This avoids invalid inputs, but it can lead to issues like poor coverage of testing and *false-negatives* or missed bugs. *TestRefineGen* mitigates this issue by invoking a generalisation procedure that weakens the encoded constraints to obtain a generator in which the under-approximation problem is reduced i.e. it can generate a super-set of the initial (under-approximate) data.

Thus, to summarize, the main contributions of this work are as follows:

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

- A novel LLM based technique *TestRefineGen* for automated test data generation in the *no-box* setting with confidential access problem and its implementation.
- An adaptation of the Pass@k metric to rank generators using LLM generated validators and select those that need repair or refinement.
- A counter-example guided refinement technique to repair generators to produce fewer invalid test inputs.
- A generalisation technique that uses LLMs to weaken generators to improve under-approximations to valid test data.
- A manually curated set of 256 textual descriptions of test data, which we call *TestRefineGen* set, based on input widgets from web-forms found in industrial scale web applications.[1]

In the next section, we describe the different components of *TestRefineGen* in detail.

II. Methodology

Figure 1 shows the functional overview of *TestRefineGen* and its constituent modules. The input to *TestRefineGen* is a set of textual descriptions of test data, where each description in the set describes the test data string for an input widget e_i in a web form and is called as the initial *context* C_i^{init} for widget e_i. The output of *TestRefineGen* is a set of Python programs, where each program in the set acts as a generator of valid test data for an input widget. *TestRefineGen* begins by applying *Rephrasing*, described below, on each initial context.

Rephrasing: It is well known that LLMs are sensitive to the prompt format and semantically similar prompts can have drastically different performance [19], [20], [21]. To account for this potential variability, we use a prompt construction technique that *rephrases* each context C_i^{init} a few times, say m, without changing its meaning. After rephrasing, for each original context C_i^{init} we end up with a set of $n = m + 1$ contexts $\mathcal{C}_i = C_i^{init} \cup \{C_i^j \mid j \in [1, m]\}$ that are semantically equivalent. An example of an initial context and a rephrased version is shown in the frame labeled "Rephrasing".

Rephrasing

Initial Context: E-Gift Voucher is a combination of 16-digit Card number and 6-digit PIN
Rephrasing prompt: Rephrase the text that I just provided such that it has the same meaning
Rephrased context: An E-Gift Voucher comprises a 16-digit Card number and a 6-digit PIN.

For each rephrased context $C_i^j \in \mathcal{C}_i$, we construct the LLM prompts P_L^{ij}, P_G^{ij}, P_S^{ij} and P_V^{ij}, described in the following and prompt the LLM in sequence with each of them.

[1]https://github.com/mygithubhk/saner2024

Priming

Context: E-Gift Voucher is a combination of 16-digit Card number and 6-digit PIN
Priming prompt: LIST all constraints on the structure of E-Gift Voucher as a numbered list. Do not give any other explanation.

Priming: The prompt P_L^{ij} is a priming prompt, which generates structured in-context information that will be given to the LLM in the subsequent synthesis steps. This follows a chain-of-thought [22] approach to improve LLM performance on the task of synthesizing generator programs. The result of the priming prompt is a numbered list of constraints that describe the test data. An example of a priming prompt is shown in the frame labeled "Priming".

Generator

Generator prompt: WRITE complete Python code to generate a string for the said E-Gift Voucher that fulfills or satisfies all constraints on the structure of said E-Gift Voucher along with all necessary libraries imports and call it from main function. Do not give any other explanation.

Synthesis (Generator): The prompt P_G^{ij} is the generator synthesis prompt and is used after the priming prompt to prompt the LLM to synthesize the generator Python program. An example of this is shown in the frame labeled "Generator". The result of the generator prompt is a Python program that generates a single test data string for the widget e_i.

Sanitisation

Sanitisation prompt: Modify this Python program such that it just generates only the E-Gift Voucher without any additional supporting text or other details. Import all the necessary libraries which are missing from original program.

Sanitisation: However, the synthesized Python generators often fail to compile because of additional accompanying text and missing imports. To address this, we introduced a sanitisation prompt P_S^{ij} to remove the extra text and add missing imports. This turns out to be a simple yet powerful technique to improve the quality of the generator programs. An example of this is shown in the frame labeled "Sanitisation".

Large Language Models (LLMs) exhibit remarkable prowess in crafting text resembling human-generated content, and their utilization has expanded to encompass code creation. Nonetheless, gauging the quality of the produced code introduces distinct hurdles. The HumanEval data-set was designed to assess the functional correctness of code generated by LLMs. It uses a metric called Pass@k [23], [13], where k code samples are generated per problem. A problem is considered solved if any sample passes the unit tests, and the total fraction of problems solved is reported.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

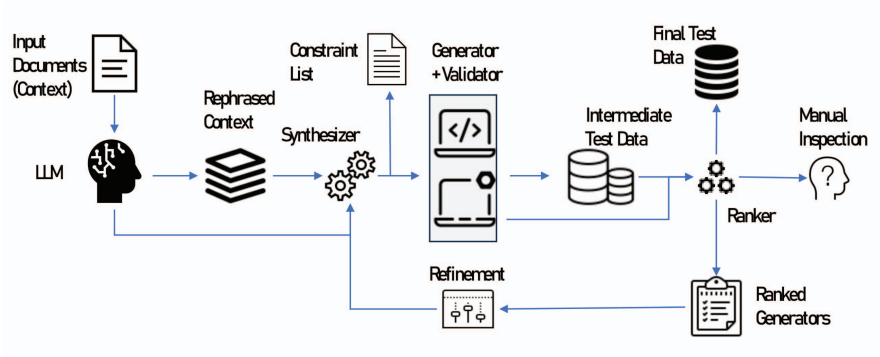

Fig. 1: Overview of *TestRefineGen*

In short, the Pass@k metric is defined as the probability that at least one of the top k generated code samples for a problem passes the unit tests given that there are c number of correct samples out of a total of n samples.

Validator

Validator prompt: WRITE Python code to accept an input string as the 1st command line parameter and validate that this given input string satisfy all constraints mentioned earlier on the structure of said E-Gift Voucher. The code should return "true" if all the constraints are satisfied, otherwise return "false". Import all necessary libraries and call it from main function. Do not give any other explanation.

Synthesis (Validator): Since, in our setting of the problem, we do not have access to a set of predefined unit tests for the Python generators synthesized by the prompts P_G^{ij} we cannot count the number of correct samples c to be able to compute an estimate of the Pass@k metric. This leads us to the classic *oracle problem* which requires us to construct a set of oracles that can validate the test data strings produced by the generators. So, in addition to the generator prompt, we also construct a validator prompt P_V^{ij} that prompts the LLM to synthesize a Python program that acts as a validator for the test data. The validator program is used to check if the generated test data is valid or not. An example of this is shown in the frame labeled "Validator". Each validator prompt is called after the corresponding priming prompt, in parallel with the generator prompt. The reason for this is to reduce validator bias by ensuring that the synthesis of the validators is not affected by in-context learning during generator synthesis. Each validator is a Python program that takes as input a test data string produced by a generator and is expected to return *true* if the string satisfies all constraints in the list obtained from the priming prompt. The validator is expected to return *false* otherwise. We note that this approach does not solve the oracle problem completely since the oracles (validator programs in this case) can themselves be imperfect and can provide incorrect judgments about the validity of the test data. However, the validators are used by *TestRefineGen* to predict potentially incorrect generator programs in combination with a ranking based approach described next.

Ranking: For each context C_i in the set $\mathcal{C}_i$, which includes the initial context C_i^{init} and its rephrased versions, the synthesis step creates a generator and a validator Python program, giving the sets of generators $\mathcal{T}_G^i$ and validators $\mathcal{T}_V^i$. Each generator program $G \in \mathcal{T}_G^i$ is now ranked as follows. It is first executed to generate a single test data string t for the widget e_i. Then, each validator in $\mathcal{T}_V^i$ is executed with t as input. The total count of the number of validators that return true on t is the rank r_G^t of the generator program G. Thus,

$$r_G^t = \mid \{V \mid V \in \mathcal{T}_V, V(t) = true\} \mid$$

The rank of a generator program G is a measure of its ability to generate valid test data. A generator G, is called a *bad* generator if it has rank $r_G^t < n$, where $n = |\mathcal{T}_V^i|$ and a *good* generator otherwise. A generator that is ranked as *bad* is scheduled for further *refinement* as described next. The objective of the refinement procedure is to improve the rank of the bad generator.

Refinement: The refinement step involves a lightweight structural analysis of the test data string t on which at least one validator program returned false. The analysis is done by calling a suite of standard Python library functions on t to obtain a set of features F_t. The suite can contain functions like: $len(t)$, $t.isalpha()$, $t.isdigit()$, $t.isalnum()$, $t.islower()$, $t.isupper()$, $t.isspace()$, $t.isdecimal()$, $t.isnumeric()$ and so on. The result of the function call on t is used to dynamically craft a prompt that supplies additional in-context information to the LLM about t. A refinement prompt is a composite prompt, containing a variable part that is dynamically constructed using the feature set F_t and a fixed part, which is a replica of the generator prompt P_G^{ij} that was used to generate t. The frame labeled "Refinement" shows two examples of refinement prompts, where Refinement Prompt 1 is constructed using the function *len*

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

and is of type length refinement, while Refinement Prompt 2 is constructed using the function $isdigit()$ and belongs to the class of datatype refinements.

Refinement

Refinement Prompt 1:The total length of E-Gift Voucher as per one of the output of the above python program is 22. Is this a valid length for the selected E-Gift Voucher? If so, output "YES" only, and do not give any other explanation. Otherwise, <*Generator prompt*>
Refinement Prompt 2: The datatype of characters allowed for the selected E-Gift Voucher is numeric, as per the provided python program. Are these types allowed according to the constraints? If so, output "YES" only, and do not give any other explanation. Otherwise, <*Generator prompt*>

The refinement step iteratively prompts the LLM with the refinement prompts to synthesize a succession of generators $G^1, G^2, \ldots, G^k$. If the result of a library function on t, added as additional in-context information, contradicts a constraint in the list of constraints obtained from the priming prompt, then the expectation is that it serves as a counterexample that directs the LLM to synthesize a new generator program that resolves this contradiction. This strategy is inspired by the CEGAR approach [17] used in formal verification of programs. A refined generator is again ranked using the validator programs and this process is iterated. The iterative refinement process is terminated when a generator program G^i that was ranked bad earlier is ranked as good or the number of iterations exceeds a predefined threshold.

In the next section, we describe an experimental evaluation of *TestRefineGen* on a set of manually curated benchmarks and present an analysis of the results.

III. Evaluation

We study the effectiveness of *TestRefineGen* in generating test data for web applications in an industrial setting when neither the application nor its source code is available due to customer confidentiality requirements.

A. Experimental Setup

For the experimental evaluation, we interacted with large industrial customers of TCS's web application testing tool to collect artifacts that can be used as input to *TestRefineGen*. Due to confidentiality requirements, we were not allowed access to either the web applications or the source code. However, we were given access to sets of documents by customers that contained information about different web-form input widgets and textual descriptions of data strings that serve as input for these widgets. We note that these documents were from a wide variety of business verticals like banking, insurance, government services, telecom, accounting, etc. Thus, we were constrained to work in the *no-box* setting described earlier. We studied these documents and used the understanding gained from them to manually scrape the internet for similar publicly available textual descriptions of data strings for web-form input widgets. Some examples of such widgets that we collected are: driving license number, bank or credit card account number, date, provider number etc. The scraped data was manually curated to remove duplicates and to ensure that the textual descriptions are relevant to the corresponding widget.

We use the Python library functions $len(t)$, $t.isdigit()$, $t.isalpha()$ and $t.isalnum()$ to obtain the feature set F_t for the refinement prompt. For the LLM we use GPT3.5 Turbo, a commercial version of GPT3 [10] that is trained on a large corpus of code and natural language. The primary reason for selecting this model was that our organization had ready access to the GPT3.5 Turbo model through an Azure OpenAI API subscription. However, since the GPT family of models from OpenAI also regularly top the leader-board for the HumanEval benchmark for code generation [24] we decided to go ahead with this model for our experiments. We used the `chat` endpoint to generate code snippets and the `temperature` parameter to control the randomness of the generated output. We set the `temperature` to 0.2 to generate more predictable output. Since, we were provided a limited budget for the Azure API, we used the `max_tokens` parameter to limit the size of the generated output to 1000 tokens. From an initial set of pilot experiments, we observed that the generator Python programs are typically small in size ($<$ 100 LOC) and hence 1000 tokens was deemed to be sufficient to generate full programs.

Through the manual scraping and curation, we were able to gather a corpus of 64 textual descriptions or initial contexts that describe data for the widgets. Given an initial context C_i we rephrased it $m = 3$ times to generate $n = 4$ contexts, by asking the LLM to do the rephrasing. The number $m = 3$ was chosen arbitrarily but with an intent to keep the LLM access cost under control. This gave us 3 variations for each original context and hence a total of 256 contexts. For our analysis we treat each of these 256 contexts as a separate input to *TestRefineGen*. Also, the $m = 3$ parameter implies that in the experimental evaluation, a generator program is bad if its rank r_G^t for a generated test data point t is less than $n = 4$, following which it is refined.

B. Observations

We now detail our observations derived from the set of experiments and later provide a statistical analysis to quantify the results. We believe that these observations can be a very useful guide for further efforts by the research and practitioner community who plan to use LLMs for automated test data generation.

Ambiguous Descriptions: When the textual description of the test data was ambiguous or incomplete, the LLM tends to hallucinate [25] and produce structural constraints that were not in the original specification. For example, on application of the priming prompt on the context C_h^1 ="An E-Gift Voucher comprises a 16-digit

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

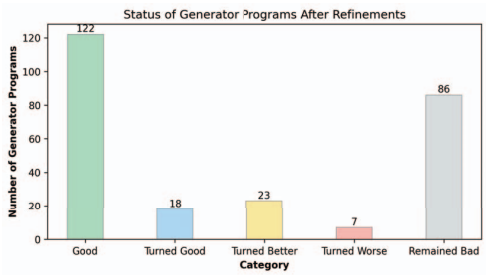

Fig. 2: Generator status before refinement

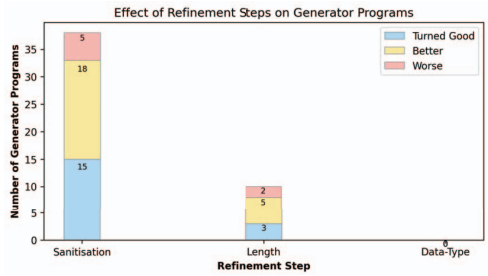

Fig. 3: Generator status breakdown after refinement

Card number and a 6-digit PIN.", the LLM generated a list of constraints that contained the property: "The Card number and PIN must be alphanumeric.", which does not correspond to an intended property of the E-Gift voucher number. Hence, the Python generator synthesized by the LLM produced voucher numbers that were incorrect e.g. "68MBLSOXOR1NXBF28HVSN3". This test input failed on a subset of validators and none of the refinement techniques was able to fix the generator program for the erroneous assumption that the voucher number should be alphanumeric. However, there was another corresponding rephrased context C_h^2 ="An E-Gift Voucher comprises a 16-digit Card number and a 6-digit PIN." that resulted in test data such as "9456405843507220-504883", which did not have alphabets but had a hyphen and caused another subset of validators to fail. In either case, we observed that it was difficult to produce correct generators and validators because the original description of the data was ambiguous about the intended properties on the structure of the data.

In another example, the original context was C_h ="The NPI provider number is a 10-digit number". The NPI provider number is a 10-digit number that is used to identify health care providers in the United States, but this information is not supplied as part of the description, making it incomplete. The LLM generated a list that contained the following constraints : "The NPI provider number is a 10-digit number. The first digit of the NPI provider number cannot be zero. The last digit of the NPI provider number is a checksum digit." In this example, the synthesized generator contained the code shown in Listing 1

Listing 1: NPI provider number (Incorrect Checksum)

```
checksum = 10 - (sum((3, 7, 1)[i % 3] * int(
    npi_provider_number[i]) for i in range(9)) %
    10)
```

for computation of a checksum for the last digit in the 10-digit number. This computation differs from the Luhn checksum algorithm actually used in the NPI provider number. A possible solution to this category of problems is to integrate the LLM with a external knowledge base that can ground it with most accurate and up-to-date facts as is done in a Retrieval Augmented Generation (RAG) [26] framework. We note that in the current implementation, we flag cases such as the ones above as potentially ambiguous or incomplete specifications and return the test data, the original context and the rephrased contexts to the user for manual inspection as shown in Figure 1.

Errors after refinement: Another behavior that we observed was that the LLM synthesized generators that were correct for either the initial or rephrased context, but failed after refinement. For example, for the context C_h ="The account number must contain 12 digits and follow the format of "999–9999999–99", with two hyphens separating the groups of numbers. It should not contain any non-numeric characters", the LLM synthesized a generator that produced test data such as "123-4567890-12", which was correct but after refinement for length and datatype, the new generator produced test data such as "123456789012", which was incorrect. The specification described the test data as a 12-digit number with two hyphens, which together imply a length of 14. The refinement for length using the Python $len()$ function prompted the LLM with a length of 14, which caused the LLM to generate test data without hyphens. In this case, the length constraint explicit in the specification was 12, while the actual expected length of the data was 14, which can be inferred by the sum of the length of the digits and the hyphens. We hypothesize that a chain-of-thought reasoning [22] prompt that guides the LLM to make inferences about the expected length of the test data would be able to fix this error.

Underapproximate generation: In some cases, the LLM produced Python generators that were capable of producing only an under-approximation of the set of all valid test data for the given textual specification. For example, for the context C_h ="Statement password is created using DOB(date of birth) and Mobile number. Statement password is numeric and should be 11 characters long. The first five characters are the last five digits of the registered mobile number. The last six characters of the password are Date of Birth in DDMMYY, that follows the constraints of valid dates", the LLM synthesized a generator that encoded Mobile number and DOB as constants as shown in Listing 2 . Thus, even though the generator produced test data that was correct, the *spread* of the test data was limited.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

Listing 2: Underapproximation of test data

```
1  def generate_statement_password(mobile_number, dob):
2      mobile_number = str(mobile_number)[-5:]
3      dob = datetime.strptime(dob, '%d-%m-%Y').
           strftime('%d%m%y')
4      statement_password = mobile_number + dob
5      return statement_password
6  if __name__ == '__main__':
7      mobile_number = 9876543210
8      dob = '01-01-2000'
9      statement_password = generate_statement_password
           (mobile_number, dob)
10     print(statement_password)
```

A poorly spread test data has low variability and diversity, with a limited range and distribution of values. This, in turn, can be a problem because it has the potential to limit the ability of the test suite to achieve a high code coverage. As a simple fix to this problem, we first executed the generator a few times and used pattern matching to identify repeating patterns in the test data. With this information, we first identified the constants in the generator program and then prompted the LLM to synthesize new generators that replace them with functions to generate random values of the same datatype. We call this a *widening* of the generators. For example, for the context C_h above, Lines 7 and 8 in the original generator were replaced by lines of code shown in Listing 3 in the widened generator.

Listing 3: Underapproximation of test data

```
1  mobile_number = random.randint
       (1000000000,9999999999)
2  day = random.randint(1, 31)
3  month = random.randint(1, 12)
4  year = random.randint(1900, 2022)
5  dob = f"{day:02d}-{month:02d}-{year:04d}"
```

The widened generator could produce test data such as "9300000111" and "17-03-2003" for the mobile number and DOB respectively, which the original generator could not produce. Although our solution is a preliminary one, we note that a simple widening of this kind can in turn lead to invalid data being generated and may lead to multiple, and even potentially unbounded, iterations of refinement. The widening technique can be made more precise by looking for invalid data during the refinement steps and asking a human expert to provide additional constraints that can be used to guide the LLM to generate better generators. Another approach that we plan to explore is to use a syntax guided synthesis (SyGUS) [27] to generate the function definitions from pre-defined templates and constraints.

Infinite loops: In rare cases, the LLM synthesized generators get stuck in an infinite loop. For example, for the context C_h ="Credit Card Number needs to be formatted in 9999-9999-9999-9999 or 9999999999999999 number formats when it is not blank. The pattern specifies: Four digits (0-9) followed by a dash. The pattern is repeated three times, deleting the last dash. It also allows an alternative pattern of 16 digits (0-9) with no dashes." the synthesized generator was as shown in Listing 4.

Listing 4: Infinite loop

```
1  def generate_credit_card_number():
2      credit_card_number = ''
3      for i in range(4):
4          credit_card_number += str(random.randint
               (1000,9999)) + '-'
5      credit_card_number = credit_card_number[:-1]
6      return credit_card_number
7  if __name__ == '__main__':
8      credit_card_number = generate_credit_card_number
           ()
9      while len(credit_card_number) != 24:
10         credit_card_number =
               generate_credit_card_number()
11     print(credit_card_number)
```

The generator program gets stuck in an infinite loop because the length of the string returned by generate_credit_card_number() is always 19, and hence the while loop condition can never be falsified.

To resolve such situations, we execute the generator programs with a timeout of 10 seconds and if it does not terminate within the timeout, we return the the original context and the rephrased contexts along with the timed-out generator program to the user for manual inspection.

C. Analysis of results

From the 64 original textual descriptions of the data or contexts, we obtained a total of 256 contexts after rephrasing. After priming, synthesis of generator and validator and sanitisation of the generator, a total of 256 Python generator programs and an equal number of validator programs were synthesized. For each generator that was synthesized for an initial context C_i^{init} or its three rephrased contexts C_i^1, C_i^2 and C_i^3, a set of $n = 4$ validators are available to validate the generated test data. Hence, following our earlier definitions, a generator G is bad if its rank $r_G^t < 4$ and good otherwise. If the rank of a generator increases or decreases after sanitisation or after one or more refinements, without becoming *good* or *bad*, we say that the refinements made it better or worse, respectively.

We found that 122 out of 256 generator programs were already *good* after the generator prompt. Another 18 out of 256 generators were *bad* after the generator step, but became *good* after sanitisation and refinement. The suite of refinements included length and data-type refinements using the Python library functions described in Section III-A. A further breakdown revealed that 15 of these 18 generators, became good after sanitisation, 3 after length refinement and 0 after data-type refinement.

There were 23 generator programs, which were bad initially but became better after all refinements were applied, but did not become *good*. Again, a breakdown shows that 18 became better after sanitisation, 5 became better after the length refinement and 0 became better after the data-type refinement. Thus, sanitisation turns

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

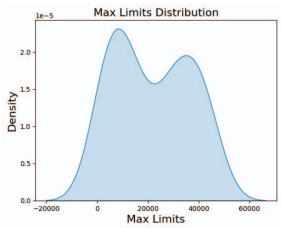

Fig. 4: 50 samples

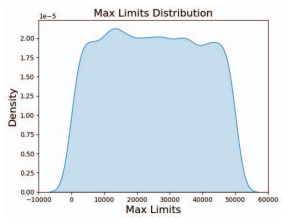

Fig. 5: 10000 samples

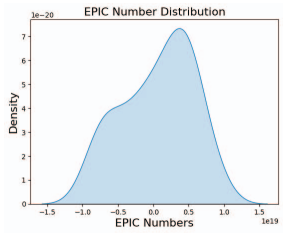

Fig. 6: 50 samples

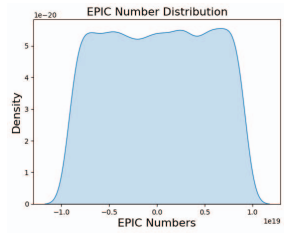

Fig. 7: 10000 samples

out to be a simple yet powerful technique that can be used to improve the rank of a generator program.

Finally, 86 generators were *bad* after the generator step and remained *bad*, with the same rank after sanitisation and refinement. There were another 7 *bad* generators that became worse, with a lower rank after sanitisation and refinements. Of these 7 generators, 5 became worse after sanitisation and 2 became worse after length refinement. The data-type refinement did not cause any generator program to become worse. We visualise this analysis of the experiments in Figure 2 and 3.

This begets the question, *does the rank of a generator program correspond to the validity of the test data that it generates?* To answer this, we manually inspected the test data generated by the 256 generator programs before and after sanitisation and refinement. During the manual inspection, we paid special attention to the intended properties of the test data and applied our judgement and domain knowledge to determine if the test data was valid or not. A generator that was found to produce invalid data and was ranked as *bad* is a true positive for us. Thus our analysis revealed the following characteristics of the 256 programs obtained after the generator synthesis step and after sanitisation and refinement and shown in the confusion matrix of Table 8. Thus, the accuracy of the ranking

	Generator ranked	Actual Valid Data	Actual Invalid Data
Before Sanitisation and Refinement	Good (122)	110 (True Negative)	12 (False Negative)
	Bad (134)	59 (False Positive)	75 (True Positive)
After Sanitisation and Refinement	Good (140)	127 (True Negative)	13 (False Negative)
	Bad (116)	70 (False Positive)	46 (True Positive)

Fig. 8: Confusion Matrix

based strategy for generator programs is 110+75/256 = 72.27% with a precision of 75/(75+59) = 55.97% and a recall of 75/(75+12) = 86.21%. After sanitisation and refinement, the accuracy of the ranking based strategy for generator programs became 127+46/256=67.57%, while the precision and recall became 46/(46+70)=39.65% and 46/(46+13)=77.9% respectively. The sanitisation and refinement steps improved the total number for generators that produced valid data from 169 to 197 i.e. from 66.01% to 76.95% for the 256 generator programs. The high recall number before and after sanitisation and refinement is a good indicator of the effectiveness of ranking generators with LLM synthesized validators. The high number of false positives and low precision is a result of the fact that the LLM synthesized validators are not perfect and can fail on valid data. The drop in precision in the latter case is due to the fact that sanitisation and refinement make generators good as well as increase the percentage of valid test data generated, while the validators continue to fail on valid data. We are exploring techniques to make our validators more robust and accurate, which in turn can improve the precision.

D. Spread of generated test data

Since, we did not have access to actual applications, we could not execute them with the generated test data to determine coverage. Instead, we hypothesize that *TestRefineGen* produces test data that is well diversified and has a high variability. If the data is well diversified with a high variability, then we expect the data to achieve high coverage, including for boundary values. To test our hypothesis, we randomly chose two contexts for which the generators were *good* after all the refinements and we manually checked that the test data being generated is valid. The first context C_1 was "*You can set maximum allowed limit up to 50000$ for Domestic & International transactions exclusively*", while the second context C_2 was "*The EPIC Number, which is printed on the front of the Voter ID card, is a 10-digit alphanumeric code comprising a combination of letters and numbers. The EPIC Number begins with a letter and ends with a digit, and it does not include any special characters or spaces. The EPIC Number only allows uppercase letters*". For these, we chose *good* generator programs P_1 and P_2, respectively from the set of generators obtained after the application of all refinements. We used these generator programs to produce 50 valid test data strings each. For the first context, the produced strings were integers, while for the second context, the produced strings were alphanumeric. The integers strings were converted to integers using the python `int` function and the alphanumeric strings were converted to integers using the python `hash` function. Using the *seaborn* library in python, we plotted the Kernel Density Estimate (KDE) plots for the two sets of data. These are shown in Figures 4 and 6. The 50 sample plots for the two contexts resemble the normal distribution, signaling perhaps a bias

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

towards data near the mean. Hence, we generated another set of 10000 valid strings for which the KDE plots (Figures 5 and 7) show a uniform distribution, indicating a good spread of the data.

To further confirm our hypothesis, we computed a pairwise normalized compression distance (NCD) [28] between different strings of each 50 test string data-set. The NCD is a measure of the similarity between two strings and is defined as follows:

$$NCD(x, y) = \frac{C(xy) - min(C(x), C(y))}{max(C(x), C(y))}$$

where $C(x), C(y)$ are the lengths of the compressed strings x, y and $C(xy)$ is the length of the compressed concatenated string xy. The Python *zlib* library was used to compress the strings. The NCD values were captured in a distance matrix and plotted as a heatmap, where hotter colors (towards yellow) indicate a higher distance and hence lower similarity, while cooler colors (towards blue) indicate a lower distance and hence higher similarity. The heatmaps for the two datasets are shown in Figures 9 and 11. We note that the heatmaps are symmetrical along the diagonal, a necessary condition for a distance metric. Both the heatmaps are *hot* indicating a high distance between the strings, which confirms that the strings are dissimilar to each other. We note that the maps remain hot even as the number of samples increase to 10000 and beyond, indicating that the strings are well diversified and have a high variability.

Another metric that we plotted was the Levenshtein distance [29] between the strings. The Levenshtein distance is a measure of the similarity between two strings and is defined as the minimum number of single character edits (insertions, deletions and substitutions) required to transform one string into another. The Levenshtein distance between two strings x and y is defined as follows:

$$L(x, y) = \begin{cases} 0, & x = y \\ \max(|x|, |y|), & x = \varnothing \text{ or } y = \varnothing \\ 1 + \min(L(x_{-1}, y), L(x, y_{-1}), & \text{otherwise} \\ \quad L(x_{-1}, y_{-1})), & \end{cases}$$

where $L(x, y_{-1})$ represents the cost of inserting a character into x, $L(x_{-1}, y)$ represents the cost of deleting a character from x and $L(x_{-1}, y_{-1})$ represents the cost of substituting a character in x. The addition of 1 represents the cost of the edit operation. The Levenshtein plots are shown in Figure 10 and 12. Again, *hotter* colors indicate a higher distance and hence lower similarity. Although, the Levenshtein plots are relatively cooler than the NCD distance plots, they still point to a high variability in the strings. This reaffirms our hypothesis that the test data generated by the LLM generated programs is well diversified and has a high variability. Thus, we believe that the test data generated by *TestRefineGen* has good potential to achieve high coverage in the challenging *no-box* testing setting. In the next section, we discuss some related work followed later by a discussion on threats to validity and future work.

IV. Related Work

There is a wide body of literature on general test data generation techniques, many of which can be applied to the domain of web-based GUI testing. Grammar-based [30], [31], [32], model-based [33], [34] and constraint-based techniques [35], [36] use grammars, models and a set of constraints, respectively to generate test data. Another set of techniques [35] and [36] uses imperative generators to generate test data. However, all these require a thorough understanding of the structure of the data and often the application, which in-turn requires a white or grey-box access to the application, which is unavailable in the *no-box* setting. Techniques that learn the grammars using machine learning [37] or synthesize them using a set of input examples and black-box access to the web-application [38] have been proposed. However, they are heavily dependent on the availability of a well curated dataset for learning, including black-box access to the application for labeling of the data and do not generalize when the input distribution changes and cannot be easily adapted to the *no-box* setting.

Random test data generation techniques typically mutate a set of high quality *seed* inputs to generate a large number of test inputs in a process called *fuzzing* [39], [40], When high quality seed inputs are not readily available they need to be generated by using constraints derived from the back-end application and a solver. In a *no-box* setting, this is not possible. As a consequence, these techniques can end up generating a large number of test inputs for which there is no a priori understanding on their validity. This means that a customer is left with the task of manually validating the generated test inputs, which can be a significant bottleneck in the testing process leading to loss of revenue and customer satisfaction.

A rich area of recent research for automated test data generation is that of mobile application testing. Although, mobile apps differ from web-based or stand-alone applications and manifest different kinds of bugs [41], there is a significant overlap with web-application testing in the type of test data generation techniques. For example, fuzzing using random generation [42] as well as using fitness metrics [43], model based [34], [33] and systematic generators [44], [45], feedback driven [46] and deep learning based [47], [11], [12] are some of the popular strategies for test input generation for mobile smartphone applications. [11] demonstrate that text or string inputs, which are characteristic of most inputs to web-based interfaces are highly prevalent in real world Android apps. Their technique uses LLMs to automatically generate text inputs for Android apps using context aware prompt generation based on text derived from the app's user interface. However, this technique does not address the issue of malformed inputs and does not provide a mechanism to repair the generated test inputs, which is an important consideration for fully or almost fully automating test data generation.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

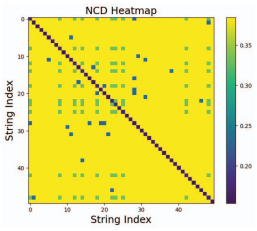

Fig. 9: NCD:Maximum Limits

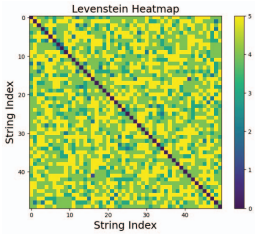

Fig. 10: Levenshtein: Maximum Limits

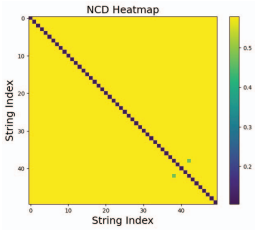

Fig. 11: NCD:EPIC Numbers

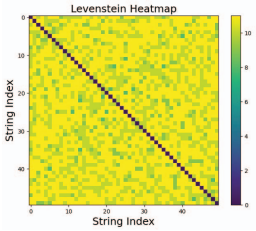

Fig. 12: Levenshtein: EPIC Numbers

TestPilot [12] changes the prompts based on failing tests to generate better tests. But, this relies on being able to execute the tests on the application. Also, the prompt refinement that attempts to generate newer tests requires white-box access to the application code or the APIs and hence is not suitable for settings that impose strict confidentiality requirements. In [48], the authors present an LLM based code synthesis technique, Jigsaw for Python Pandas API that uses carefully crafted syntactic transformations and search based methods to repair incorrectly synthesized code. Thus, although there is some overlap in the general approach used in *TestRefineGen*, there are important differences. The primary difference is that Jigsaw does not directly address the issue of generating test inputs from natural language descriptions and hence it is difficult to evaluate its effectiveness in this setting. Also, *TestRefineGen* uses a ranking based approach to evaluate potentially correct programs, which is not used in Jigsaw.

V. Discussion

A. Threats to validity

The primary threat to validity of *TestRefineGen* comes from the use of LLM synthesized validators for ranking and shortlisting generator programs for refinement. As discussed in Section II and Section III-C, the LLM synthesized validators are not perfect and may result in both false positives and false negatives. We are developing a technique that can generate more accurate validators and mitigate this threat. Another threat to validity of *TestRefineGen* is that we do not have access to the actual applications cannot determine the code coverage on the produced test data. This is a fundamental limitation of the *no-box* setting. However, a mitigation strategy already under development for this threat is to focus on extracting business rules from textual documents, which potentially mirror application logic and synthesize generators that produce test data to *cover* these business rules [49]. A potential threat to validity of *TestRefineGen* could arise if a customer is not able to provide documents containing textual description of the test data. However, we plan to build a repository of such documents for common applications as well as present a text input interface to allow customers to provide textual descriptions of the test data, which can help to manage such situations.

B. Future Work

There are several avenues for future work. As a first step, we are in the process of converting *TestRefineGen* into a plugin module for the TCS industrial scale automated testing tool. *TestRefineGen* was architectured with an independent workflow and a well defined API to facilitate this integration. As a next step, we would like to extend *TestRefineGen* to generate test data for applications that have a more complex input space, e.g. inputs that are related to each other by constraints described as business rules. Another line of work currently being explored is to develop a ranking for generators that is based on the semantic similarity between the generated test data and the textual description of the test data.

VI. Conclusion

In this paper, we introduced *TestRefineGen*, a counter example-guided strategy to automatically generate test data for *no-box* web-application testing from textual descriptions of the test data. Thus, it fulfills a niche need in the industry for a scalable and cost effective solution to generate test data for web-testing applications in settings where confidentiality and privacy concerns prevent access to customer applications or source code. The key idea behind *TestRefineGen* is to use a combination of LLMs and generator programs to generate test data, which allows easy packaging of the technique as a plugin module for existing industrial scale automated testing tool, while keeping data generation cost overruns due to LLM usage under control. The novel strategy of using LLM synthesized validators and ranking functions to validate and identify potentially incorrect generator programs for counter-example guided repair is a key contribution that differentiates *TestRefineGen* from existing techniques. This makes *TestRefineGen* resilient to the limitations of LLMs, while still allowing it to leverage their power to generate test data from textual descriptions. Thus *TestRefineGen* is an efficient, robust and cost effective technique for automatically generating test data for web-applications at industrial scale.

References

[1] A. Mesbah, "Software Analysis for the Web: Achievements and Prospects," in *2016 IEEE 23rd International Conference on*

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

Software Analysis, Evolution, and Reengineering (SANER), vol. 5, Mar. 2016, pp. 91–103.

[2] E. Daka and G. Fraser, "A Survey on Unit Testing Practices and Problems," in *2014 IEEE 25th International Symposium on Software Reliability Engineering*, Nov. 2014, pp. 201–211, iSSN: 2332-6549.

[3] M. Leotta, D. Clerissi, F. Ricca, and P. Tonella, "Chapter Five - Approaches and Tools for Automated End-to-End Web Testing," in *Advances in Computers*, A. Memon, Ed. Elsevier, Jan. 2016, vol. 101, pp. 193–237. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0065245815000686

[4] F. Ricca and A. Stocco, "Web Test Automation: Insights from the Grey Literature," in *SOFSEM 2021: Theory and Practice of Computer Science*, ser. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2021, pp. 472–485.

[5] S. Berner, R. Weber, and R. K. Keller, "Observations and lessons learned from automated testing," in *Proceedings of the 27th international conference on Software engineering*, ser. ICSE '05. New York, NY, USA: Association for Computing Machinery, May 2005, pp. 571–579. [Online]. Available: https://doi.org/10.1145/1062455.1062556

[6] "Tata Consultancy Services: Driving Innovation and Building on Belief." [Online]. Available: https://www.tcs.com/home-page

[7] S. Agrawal, R. Venkatesh, U. Shrotri, A. Zare, and S. Verma, "Scaling Test Case Generation For Expressive Decision Tables," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, Oct. 2020, pp. 364–374, iSSN: 2159-4848. [Online]. Available: https://ieeexplore.ieee.org/document/9159105

[8] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 7871–7880. [Online]. Available: https://aclanthology.org/2020.acl-main.703

[9] P. Bareiß, B. Souza, M. d'Amorim, and M. Pradel, "Code Generation Tools (Almost) for Free? A Study of Few-Shot, Pre-Trained Language Models on Code," Jun. 2022, arXiv:2206.01335 [cs]. [Online]. Available: http://arxiv.org/abs/2206.01335

[10] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS'20. Red Hook, NY, USA: Curran Associates Inc., Dec. 2020, pp. 1877–1901.

[11] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, "Fill in the Blank: Context-aware Automated Text Input Generation for Mobile GUI Testing," Dec. 2022, arXiv:2212.04732 [cs]. [Online]. Available: http://arxiv.org/abs/2212.04732

[12] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "Adaptive Test Generation Using a Large Language Model," Feb. 2023, arXiv:2302.06527 [cs]. [Online]. Available: http://arxiv.org/abs/2302.06527

[13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," Jul. 2021, arXiv:2107.03374 [cs]. [Online]. Available: http://arxiv.org/abs/2107.03374

[14] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?" in *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, A. Korhonen, D. R. Traum, and L. Màrquez, Eds. Association for Computational Linguistics, 2019, pp. 4791–4800. [Online]. Available: https://doi.org/10.18653/v1/p19-1472

[15] P. Sawadpong, E. B. Allen, and B. J. Williams, "Exception Handling Defects: An Empirical Study," in *2012 IEEE 14th International Symposium on High-Assurance Systems Engineering*, Oct. 2012, pp. 90–97, iSSN: 1530-2059.

[16] C. Yilmaz, E. Dumlu, M. B. Cohen, and A. Porter, "Reducing Masking Effects in CombinatorialInteraction Testing: A Feedback DrivenAdaptive Approach," *IEEE Transactions on Software Engineering*, vol. 40, no. 1, pp. 43–66, Jan. 2014. [Online]. Available: https://doi.org/10.1109/TSE.2013.53

[17] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-Guided Abstraction Refinement," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, E. A. Emerson and A. P. Sistla, Eds. Berlin, Heidelberg: Springer, 2000, pp. 154–169.

[18] K. Fögen and H. Lichter, "Generation of Invalid Test Inputs from Over-Constrained Test Models for Combinatorial Robustness Testing," in *2020 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, Oct. 2020, pp. 171–180.

[19] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large Language Models are Zero-Shot Reasoners," Jan. 2023, arXiv:2205.11916 [cs]. [Online]. Available: http://arxiv.org/abs/2205.11916

[20] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large Language Models Are Human-Level Prompt Engineers," Mar. 2023, arXiv:2211.01910 [cs]. [Online]. Available: http://arxiv.org/abs/2211.01910

[21] T. Zhang, X. Wang, D. Zhou, D. Schuurmans, and J. E. Gonzalez, "TEMPERA: TEST-TIME PROMPT EDITING VIA REIN- FORCEMENT LEARNING," 2023.

[22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," Jan. 2023, arXiv:2201.11903 [cs]. [Online]. Available: http://arxiv.org/abs/2201.11903

[23] S. Kulal, P. Pasupat, K. Chandra, M. Lee, O. Padon, A. Aiken, and P. S. Liang, "SPoC: Search-based Pseudocode to Code," in *Advances in Neural Information Processing Systems*, vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/hash/7298332f04ac004a0ca44cc69ecf6f6b-Abstract.html

[24] paperswithcode, "Papers with Code - HumanEval Benchmark (Code Generation)." [Online]. Available: https://paperswithcode.com/sota/code-generation-on-humaneval

[25] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models," Sep. 2023, arXiv:2309.01219 [cs]. [Online]. Available: http://arxiv.org/abs/2309.01219

[26] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Apr. 2021, arXiv:2005.11401 [cs] version: 4. [Online]. Available: http://arxiv.org/abs/2005.11401

[27] R. Alur, R. Bodik, G. Juniwal, M. M. K. Martin, M. Raghothaman, S. A. Seshia, R. Singh, A. Solar-Lezama, E. Torlak, and A. Udupa, "Syntax-guided synthesis," in *2013 Formal Methods in Computer-Aided Design*, Oct. 2013, pp. 1–8. [Online]. Available: https://ieeexplore.ieee.org/document/6679385

[28] A. R. Cohen and P. M. Vitányi, "Normalized Compression Distance of Multisets with Applications," *IEEE Transactions*

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.

on Pattern Analysis and Machine Intelligence, vol. 37, no. 8, pp. 1602–1614, Aug. 2015, conference Name: IEEE Transactions on Pattern Analysis and Machine Intelligence. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/6967789

[29] V. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet physics doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[30] P. Godefroid, A. Kiezun, and M. Y. Levin, "Grammar-based whitebox fuzzing," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '08. New York, NY, USA: Association for Computing Machinery, Jun. 2008, pp. 206–215. [Online]. Available: https://doi.org/10.1145/1375581.1375607

[31] R. Hodovan, A. Kiss, and T. Gyimothy, "Grammarinator: a grammar-based open source fuzzer," in *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, ser. A-TEST 2018. New York, NY, USA: Association for Computing Machinery, Nov. 2018, pp. 45–48. [Online]. Available: https://doi.org/10.1145/3278186.3278193

[32] D. Steinhöfel and A. Zeller, "Input invariants," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, Nov. 2022, pp. 583–594. [Online]. Available: https://doi.org/10.1145/3540250.3549139

[33] N. P. Borges Jr., J. Hotzkow, and A. Zeller, "DroidMate-2: a platform for Android test generation," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. Montpellier France: ACM, Sep. 2018, pp. 916–919. [Online]. Available: https://dl.acm.org/doi/10.1145/3238147.3240479

[34] Y. Li, Z. Yang, Y. Guo, and X. Chen, "DroidBot: a lightweight UI-Guided test input generator for android," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, May 2017, pp. 23–26.

[35] K. Claessen and J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," *ACM SIGPLAN Notices*, vol. 35, no. 9, pp. 268–279, Sep. 2000. [Online]. Available: https://dl.acm.org/doi/10.1145/357766.351266

[36] B. Daniel, D. Dig, K. Garcia, and D. Marinov, "Automated testing of refactoring engines," in *Proceedings of the the 6th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*, ser. ESEC-FSE '07. New York, NY, USA: Association for Computing Machinery, Sep. 2007, pp. 185–194. [Online]. Available: https://doi.org/10.1145/1287624.1287651

[37] P. Godefroid, H. Peleg, and R. Singh, "Learn&Fuzz: Machine learning for input fuzzing," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Oct. 2017, pp. 50–59.

[38] O. Bastani, R. Sharma, A. Aiken, and P. Liang, "Synthesizing program input grammars," *ACM SIGPLAN Notices*, vol. 52, no. 6, pp. 95–110, Sep. 2017. [Online]. Available: https://dl.acm.org/doi/10.1145/3140587.3062349

[39] M. Sutton, A. Greene, and P. Amini, *Fuzzing: Brute Force Vulnerability Discovery*. Addison-Wesley Professional, Jun. 2007.

[40] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of UNIX utilities," *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, Dec. 1990. [Online]. Available: https://dl.acm.org/doi/10.1145/96267.96279

[41] C. Hu and I. Neamtiu, "Automating GUI testing for Android applications," in *Proceedings of the 6th International Workshop on Automation of Software Test*, ser. AST '11. New York, NY, USA: Association for Computing Machinery, May 2011, pp. 77–83. [Online]. Available: https://doi.org/10.1145/1982595.1982612

[42] "UI/Application Exerciser Monkey Android Studio," Jun. 2023. [Online]. Available: https://developer.android.com/studio/test/other-testing-tools/monkey

[43] R. Sasnauskas and J. Regehr, "Intent fuzzer: crafting intents of death," in *Proceedings of the 2014 Joint International Workshop on Dynamic Analysis (WODA) and Software and System Performance Testing, Debugging, and Analytics (PERTEA)*, ser. WODA+PERTEA 2014. New York, NY, USA: Association for Computing Machinery, Jul. 2014, pp. 1–5. [Online]. Available: https://doi.org/10.1145/2632168.2632169

[44] R. Mahmood, N. Mirzaei, and S. Malek, "EvoDroid: segmented evolutionary testing of Android apps," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014. New York, NY, USA: Association for Computing Machinery, Nov. 2014, pp. 599–609. [Online]. Available: https://doi.org/10.1145/2635868.2635896

[45] N. Mirzaei, J. Garcia, H. Bagheri, A. Sadeghi, and S. Malek, "Reducing combinatorics in GUI testing of android applications," in *Proceedings of the 38th International Conference on Software Engineering*. Austin Texas: ACM, May 2016, pp. 559–570. [Online]. Available: https://dl.acm.org/doi/10.1145/2884781.2884853

[46] Y. He, L. Zhang, Z. Yang, Y. Cao, K. Lian, S. Li, W. Yang, Z. Zhang, M. Yang, Y. Zhang, and H. Duan, "TextExerciser: Feedback-driven Text Input Exercising for Android Applications," in *2020 IEEE Symposium on Security and Privacy (SP)*, May 2020, pp. 1071–1087, iSSN: 2375-1207.

[47] P. Liu, X. Zhang, M. Pistoia, Y. Zheng, M. Marques, and L. Zeng, "Automatic text input generation for mobile testing," in *Proceedings of the 39th International Conference on Software Engineering*, ser. ICSE '17. Buenos Aires, Argentina: IEEE Press, May 2017, pp. 643–653. [Online]. Available: https://doi.org/10.1109/ICSE.2017.65

[48] N. Jain, S. Vaidyanath, A. Iyer, N. Natarajan, S. Parthasarathy, S. Rajamani, and R. Sharma, "Jigsaw: large language models meet program synthesis," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, Jul. 2022, pp. 1219–1231. [Online]. Available: https://doi.org/10.1145/3510003.3510203

[49] S. H. Jensen, S. Thummalapenta, S. Sinha, and S. Chandra, "Test Generation from Business Rules," in *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*, Apr. 2015, pp. 1–10, iSSN: 2159-4848.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:54:26 UTC from IEEE Xplore. Restrictions apply.